

P-sweep: vanilla vs orthogonal AL-RNN on rotated Lorenz63

Goal

Compare two AL-RNN variants on a randomly rotated Lorenz63 attractor, sweeping the number of ReLU units $P = 2..6$.

Architectures

Both models have state z in $\mathbb{R}^M$ ($M=20$), observation x in $\mathbb{R}^N$ ($N=3$).
Shared parameters: A (diag skip), W ($M \times M$ coupling), h (bias), B ($N \times M$ input embedding). Only the last P hidden units pass through a ReLU; the others stay linear.

Vanilla: $z_{t+1} = A \cdot z_t + \text{phi}_P(z_t) W^T + h$
Orthogonal: $z_{t+1} = A \cdot z_t + \text{phi}_P(Q z_t + b_Q) W^T + h$
with Q in $O(M)$, enforced by
`torch.nn.utils.parametrizations.orthogonal`

Training protocol

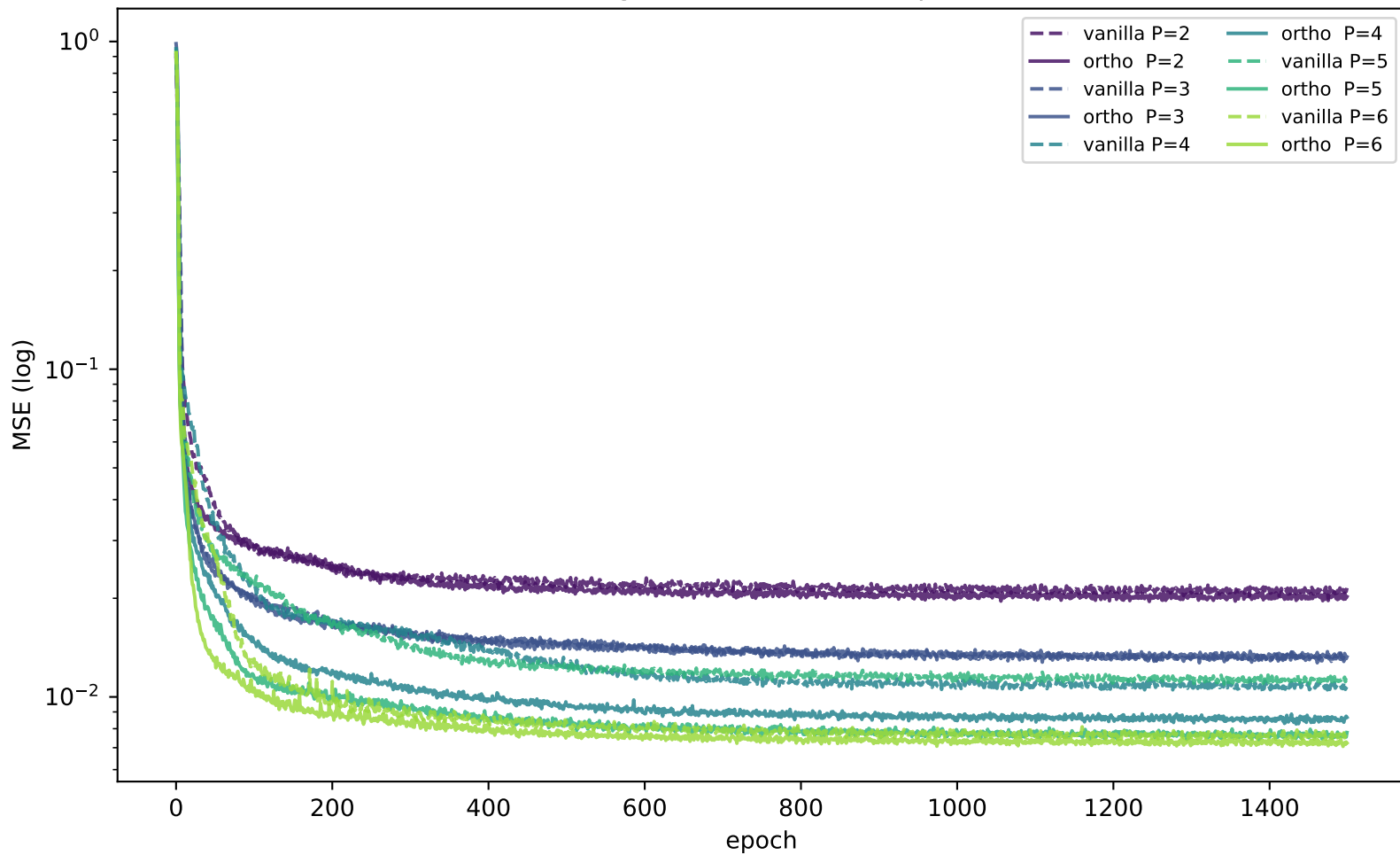
data : Lorenz63 train/test, first 500 steps dropped,
rotated in $\mathbb{R}^3$ by a fixed random orthogonal
 Q_{dataset} (72 deg, $\det=+1$, seed=42)
 M, N : 20, 3
 P : [2, 3, 4, 5, 6]
seq length : 128
batch size : 64
batches/ep : 20
epochs : 1500
optimiser : Adam, lr $1e-3 \rightarrow 1e-5$ exponential decay
loss : MSE on first $N=3$ hidden coordinates
teacher forcing: hard reset at $t=0$, $\alpha=1$ injection
every $n_{\text{interleave}}=16$ steps
free-run test : 5000-step rollout from $X_{\text{test_trans}}[0]$

Speed-ups vs the original notebook: cache the orthogonal-parametrisation output once per sequence (avoids recomputing on each of 128 inner steps), `torch.cat` split instead of `clone+assign`, batch 32 $\rightarrow$ 64. Full sweep ran in ~98 minutes on 4 CPUs (projected ~3.5 h before these changes).

Results (final training MSE)

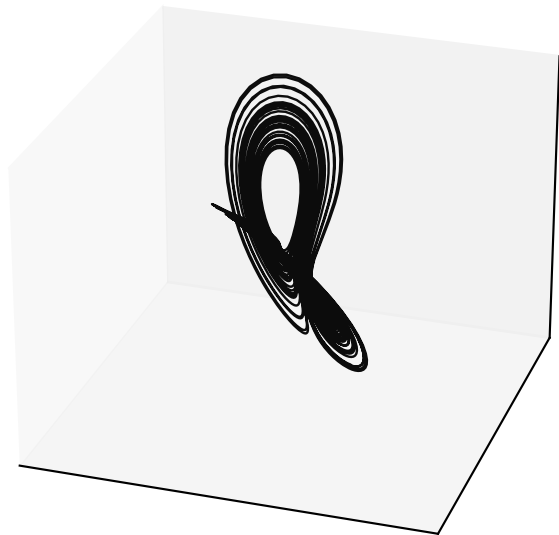
P	vanilla	orthogonal	rel. gap
2	0.02150	0.02023	-5.9%

Training loss curves, 1500 epochs

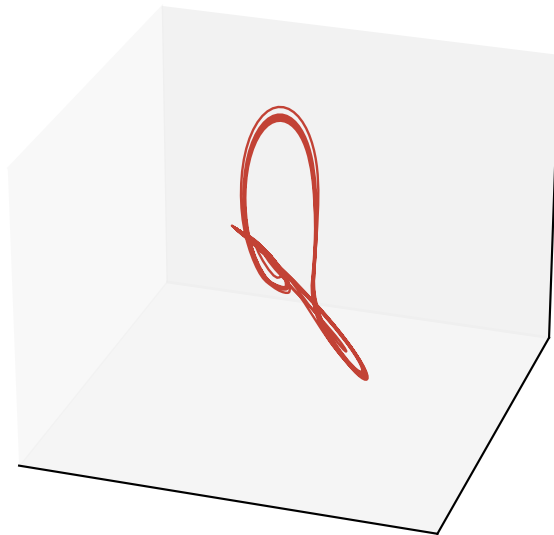


Free-run reconstruction at $P = 2$ (3-D view, 5000 steps)

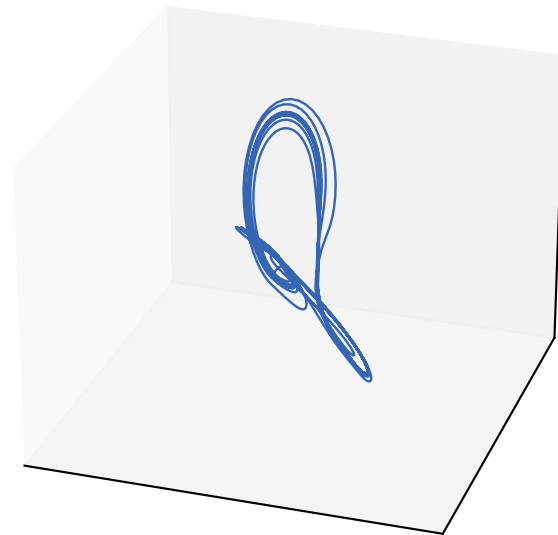
Ground truth (rotated)



Vanilla AL-RNN

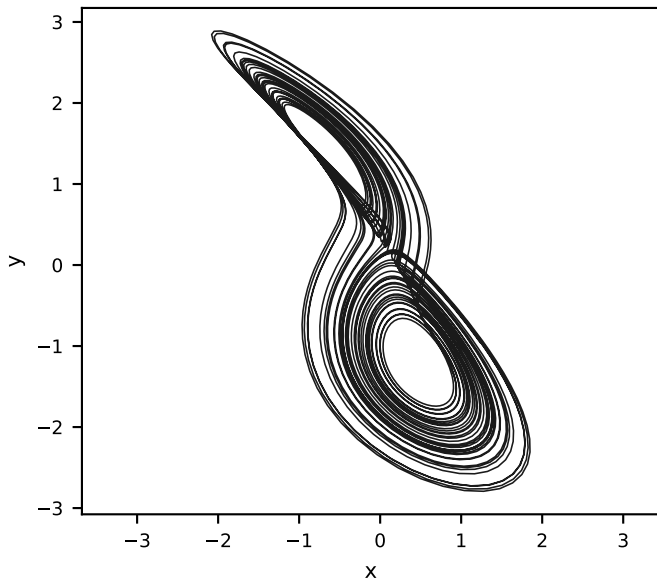


Orthogonal AL-RNN

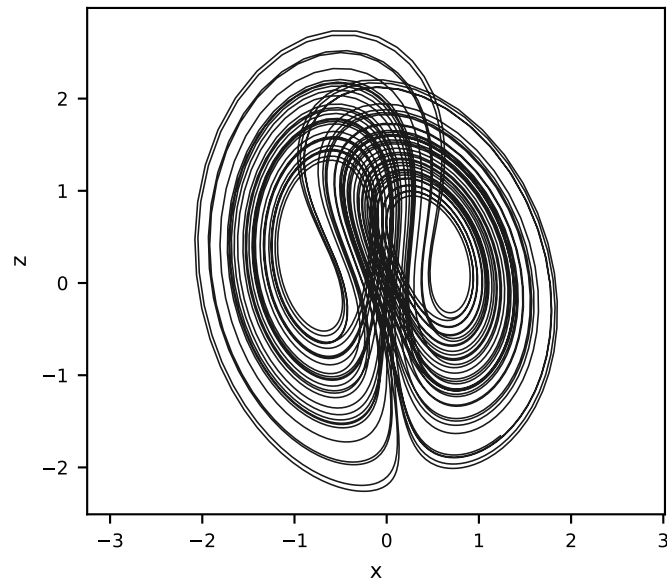


Free-run reconstruction at $P = 2$ (2-D projections, 5000 steps)

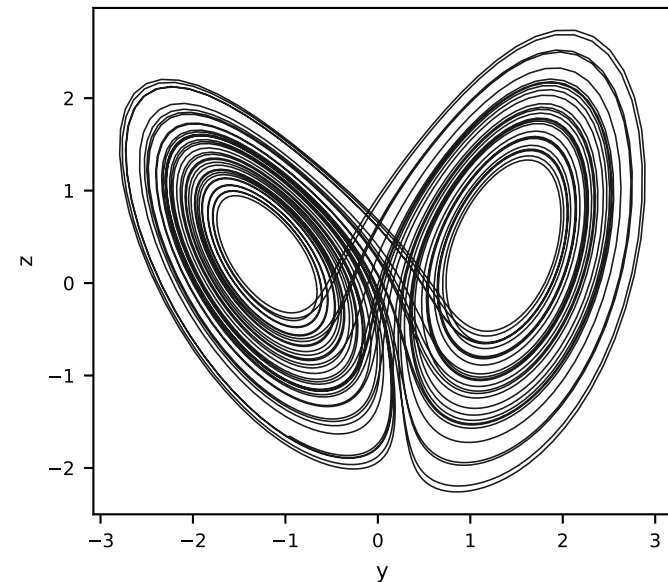
Ground truth x-y



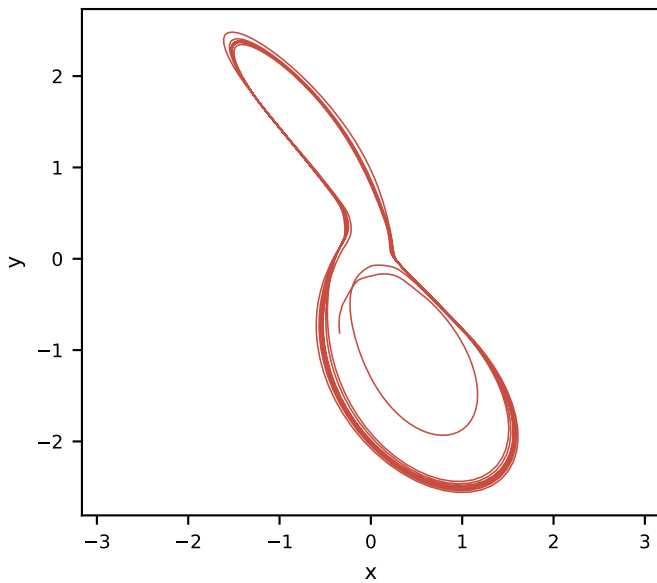
Ground truth x-z



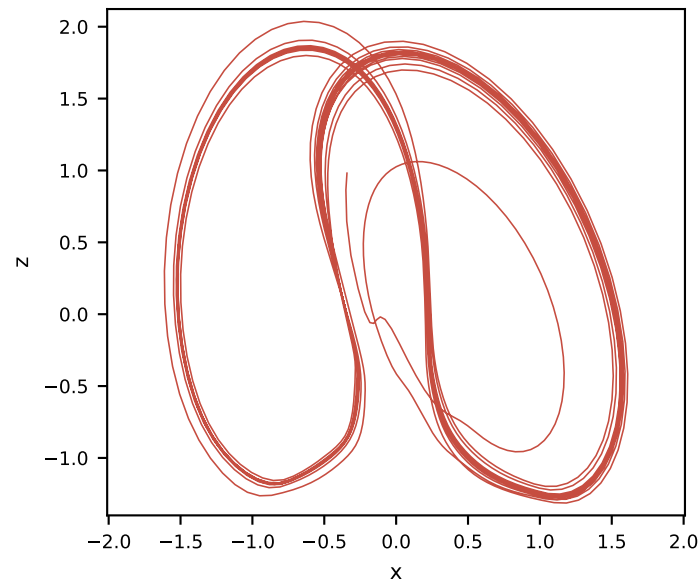
Ground truth y-z



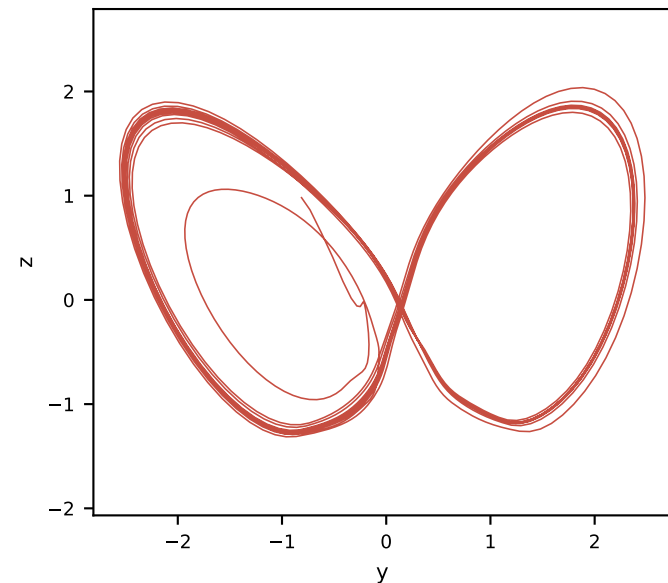
Vanilla x-y



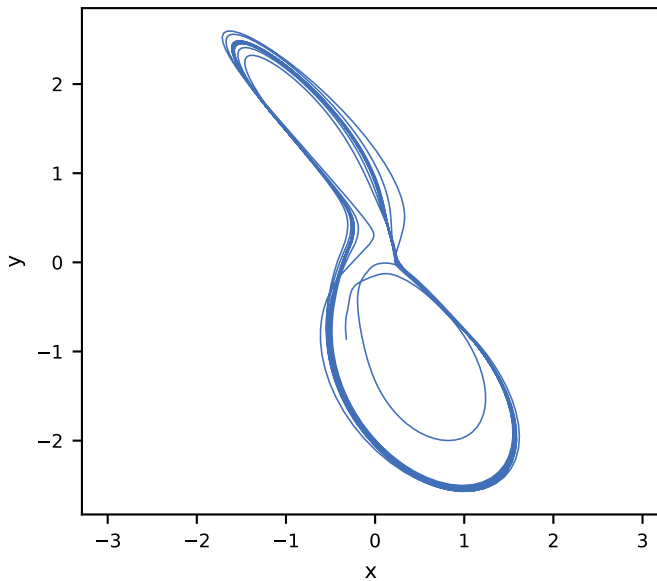
Vanilla x-z



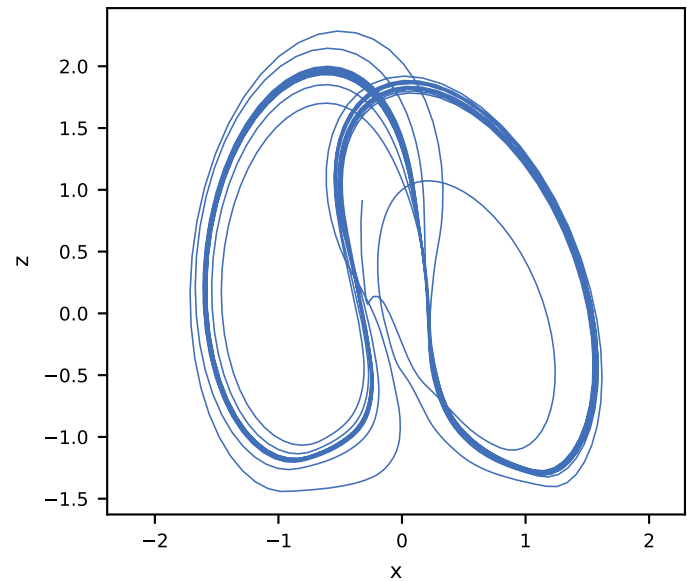
Vanilla y-z



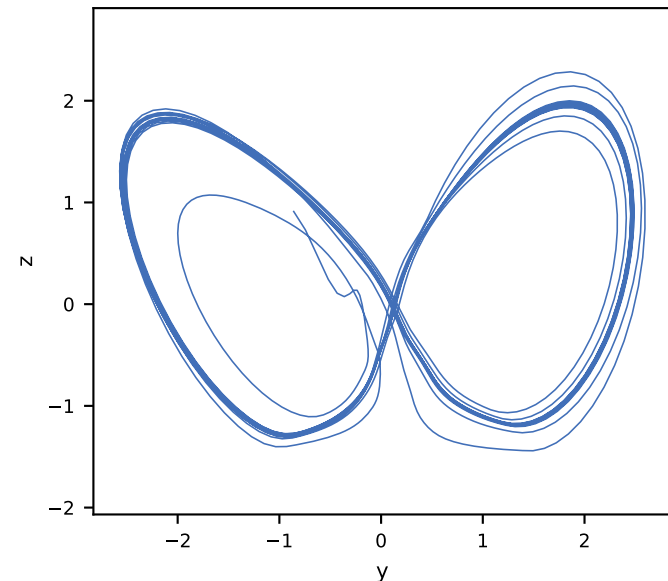
Orthogonal x-y



Orthogonal x-z

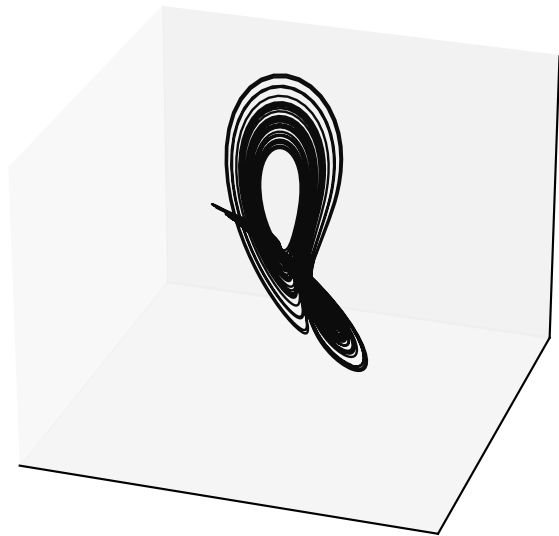


Orthogonal y-z

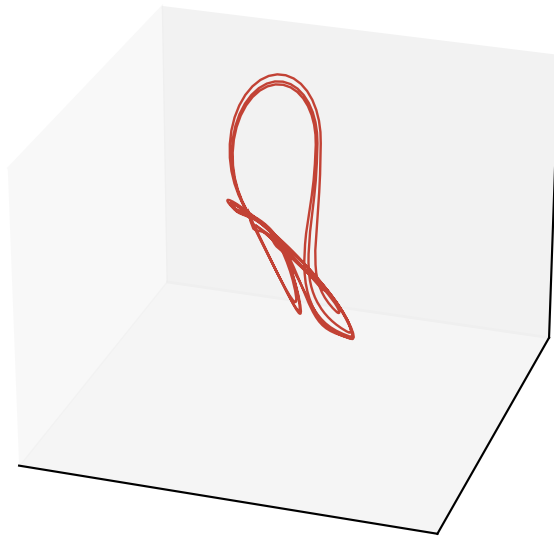


Free-run reconstruction at $P = 3$ (3-D view, 5000 steps)

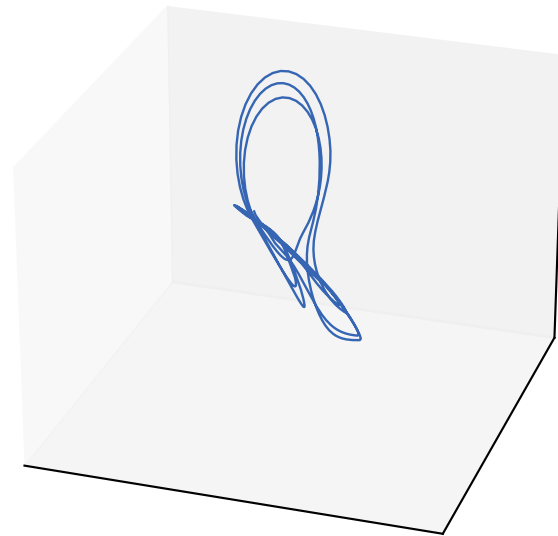
Ground truth (rotated)



Vanilla AL-RNN

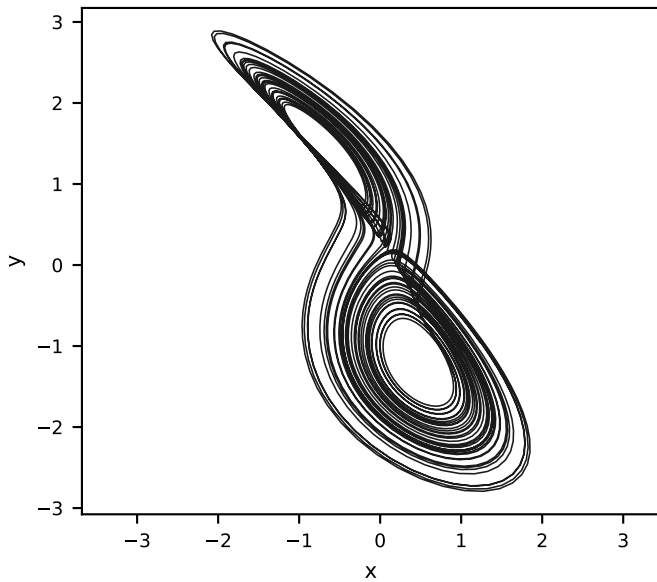


Orthogonal AL-RNN

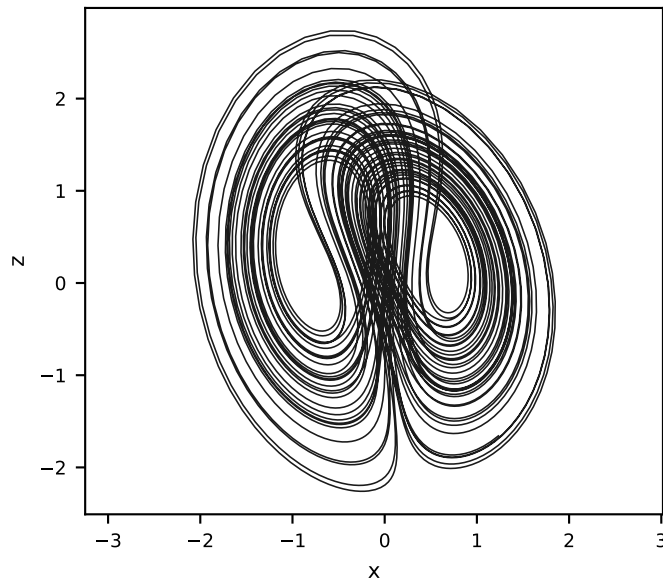


Free-run reconstruction at $P = 3$ (2-D projections, 5000 steps)

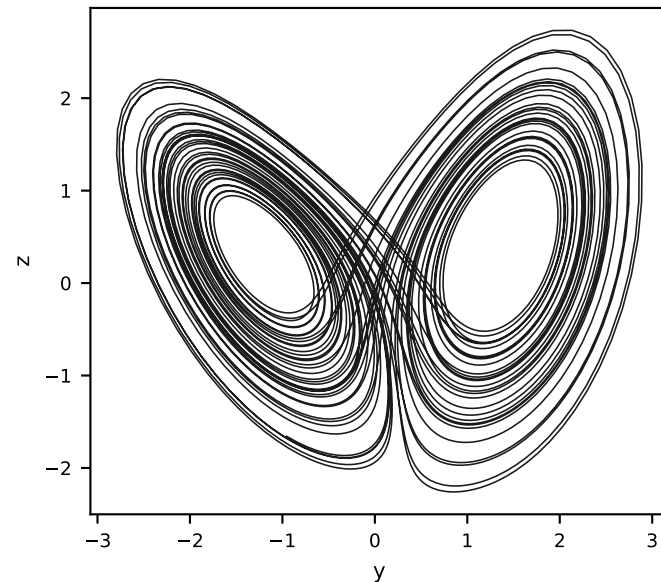
Ground truth x-y



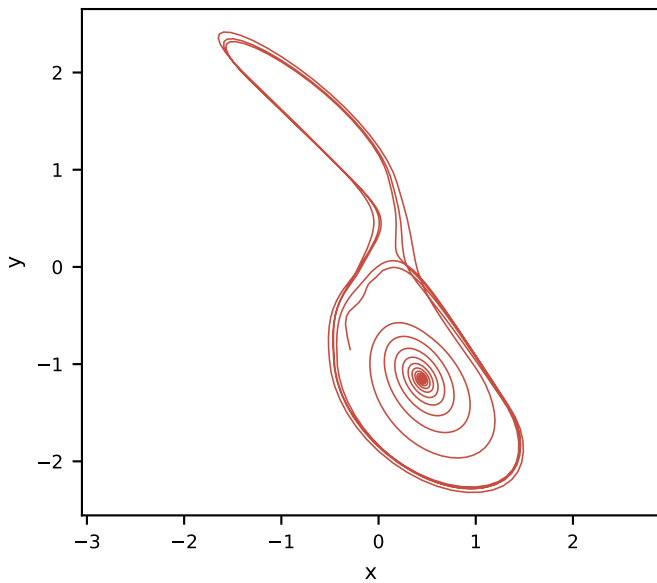
Ground truth x-z



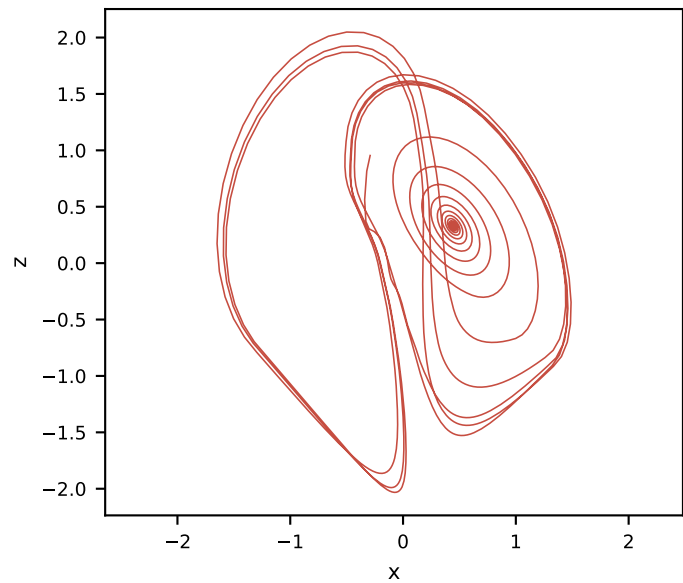
Ground truth y-z



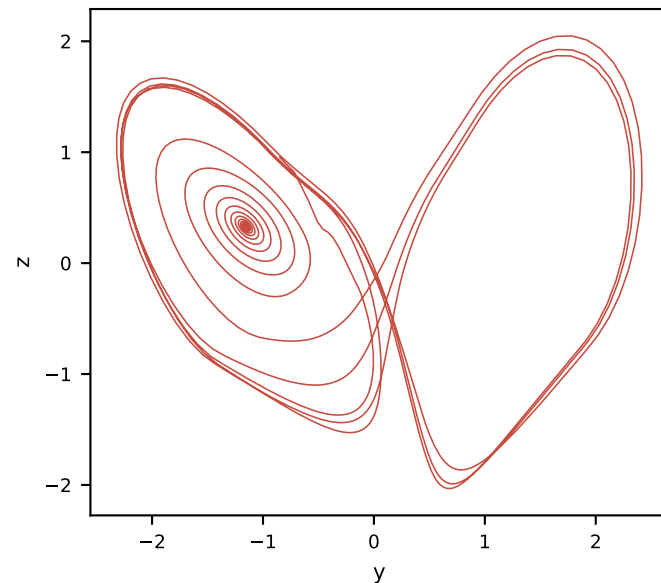
Vanilla x-y



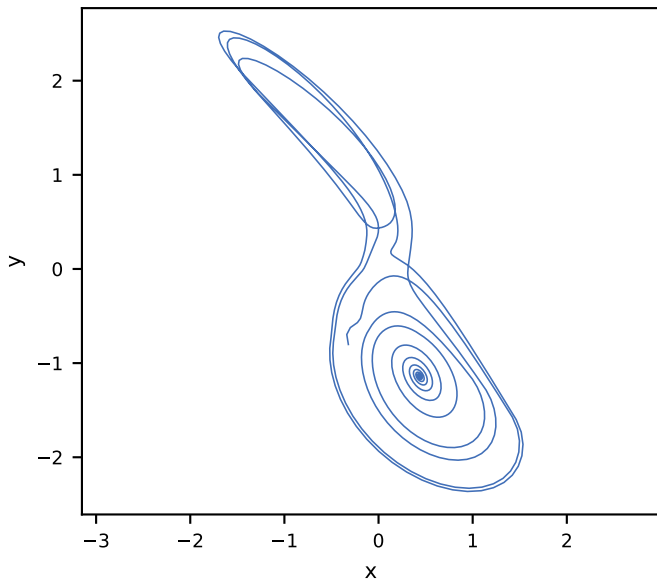
Vanilla x-z



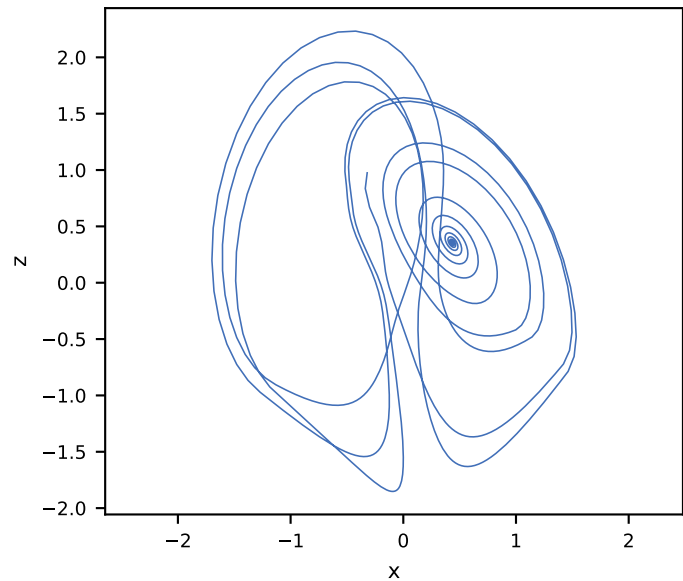
Vanilla y-z



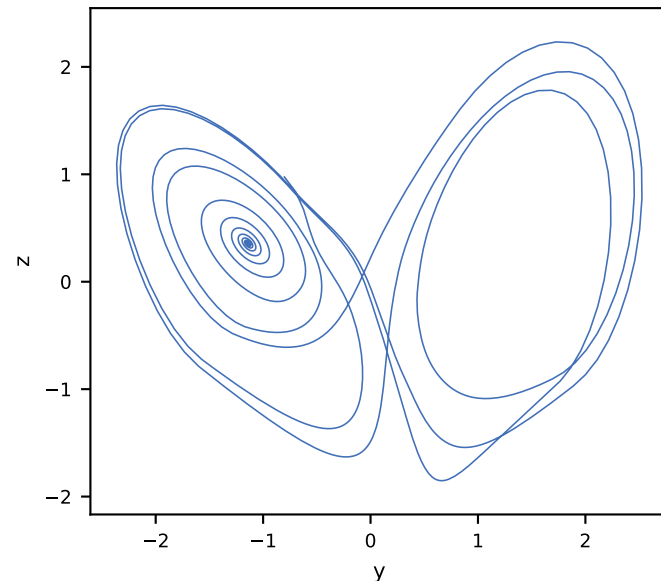
Orthogonal x-y



Orthogonal x-z

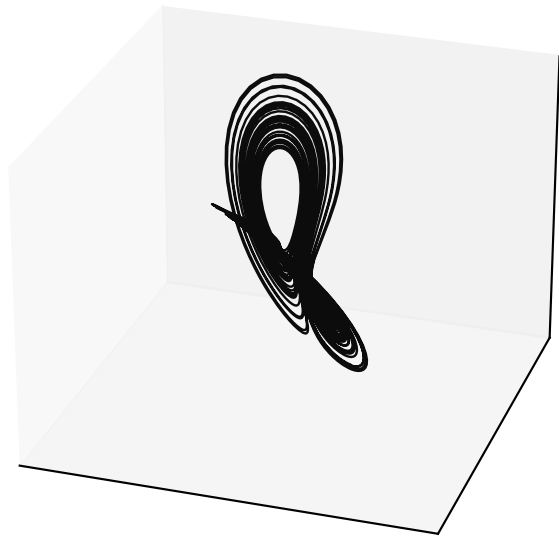


Orthogonal y-z

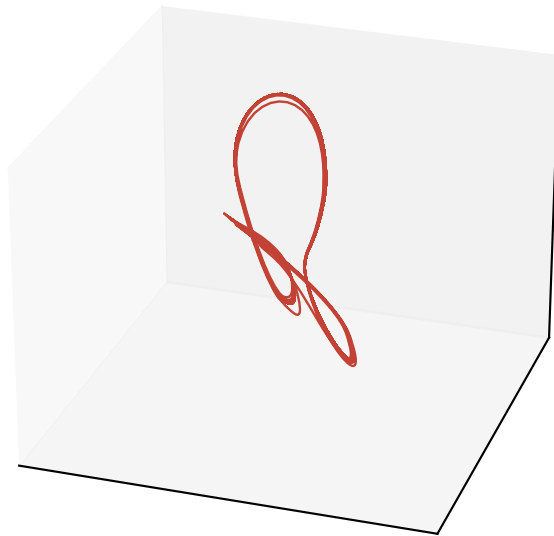


Free-run reconstruction at $P = 4$ (3-D view, 5000 steps)

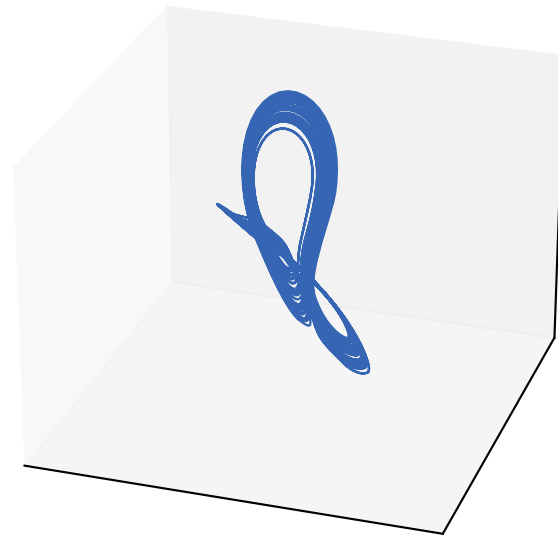
Ground truth (rotated)



Vanilla AL-RNN

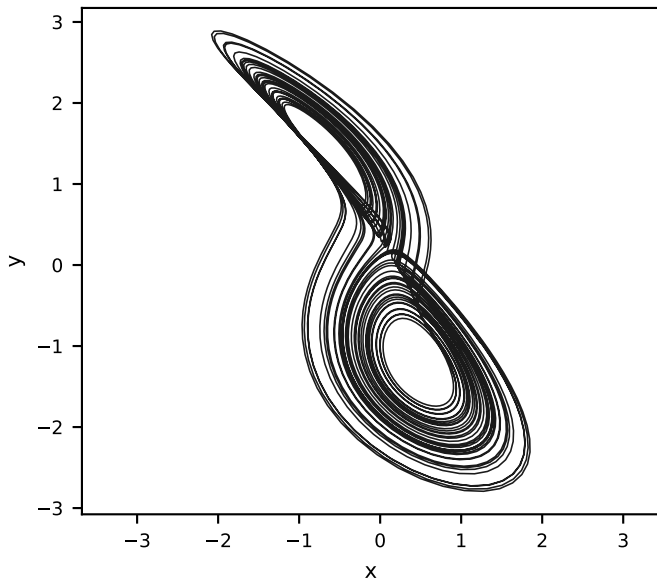


Orthogonal AL-RNN

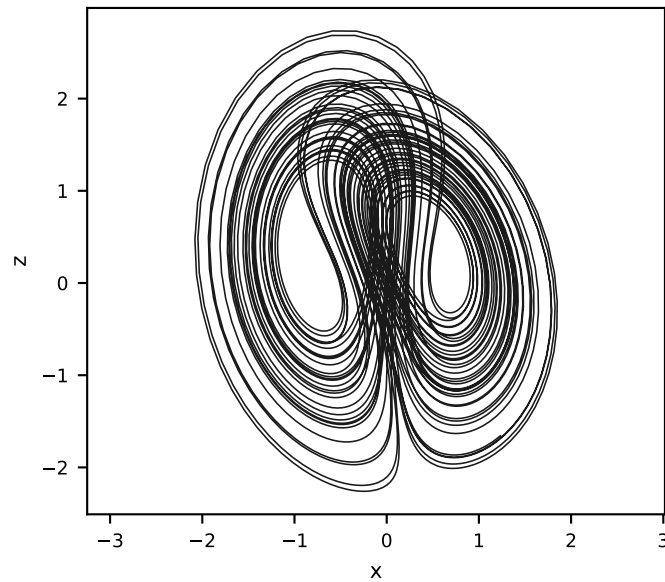


Free-run reconstruction at $P = 4$ (2-D projections, 5000 steps)

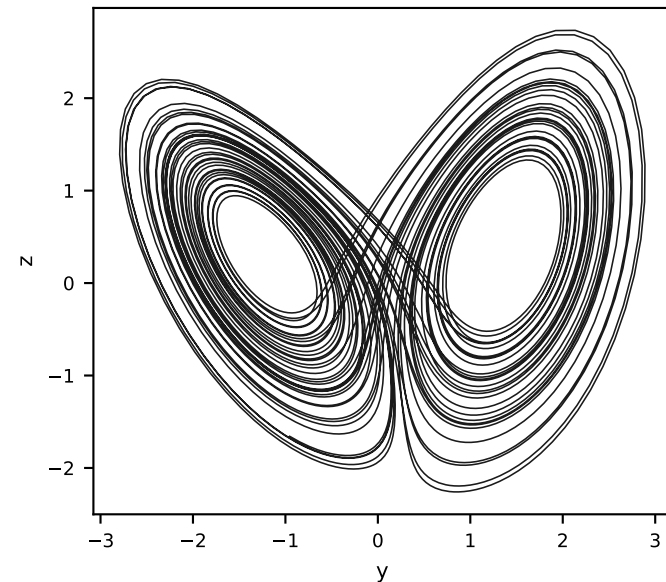
Ground truth x-y



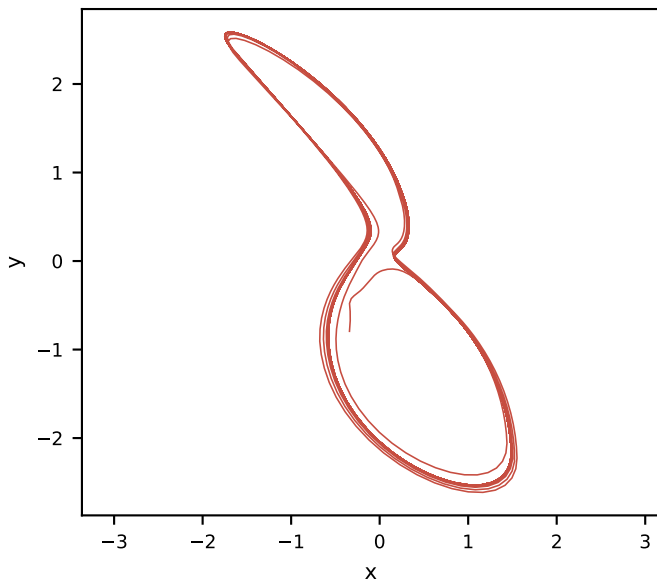
Ground truth x-z



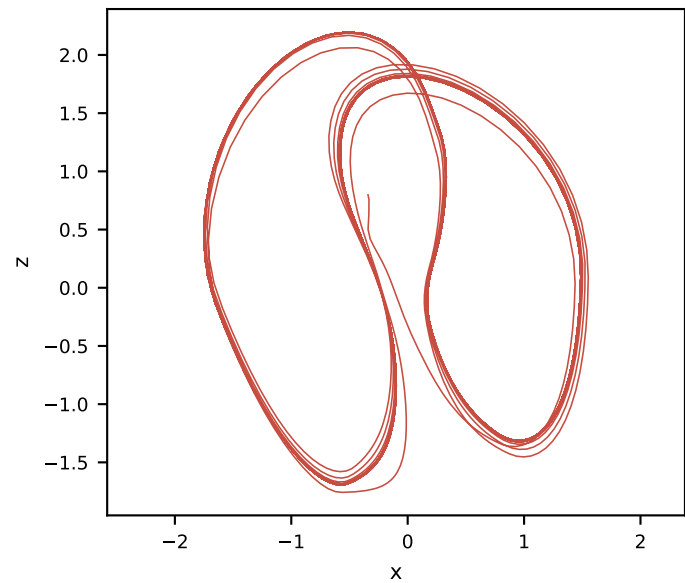
Ground truth y-z



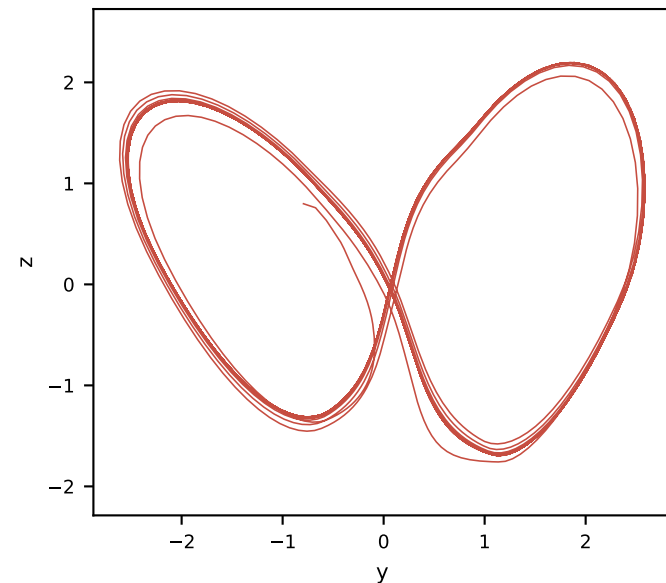
Vanilla x-y



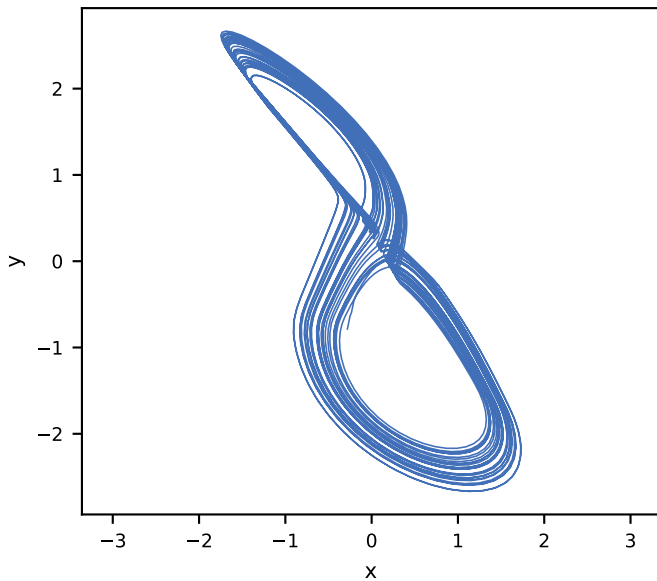
Vanilla x-z



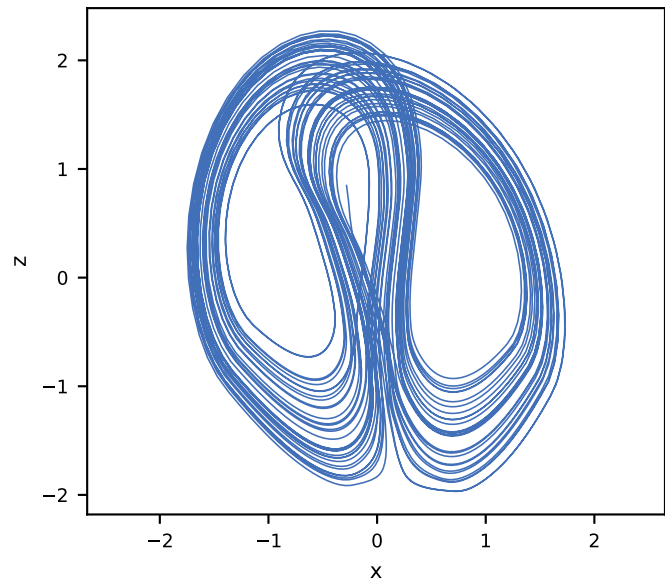
Vanilla y-z



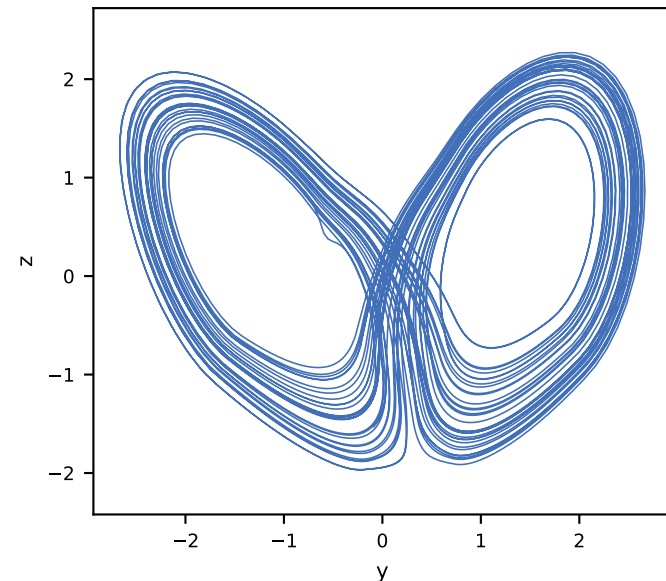
Orthogonal x-y



Orthogonal x-z

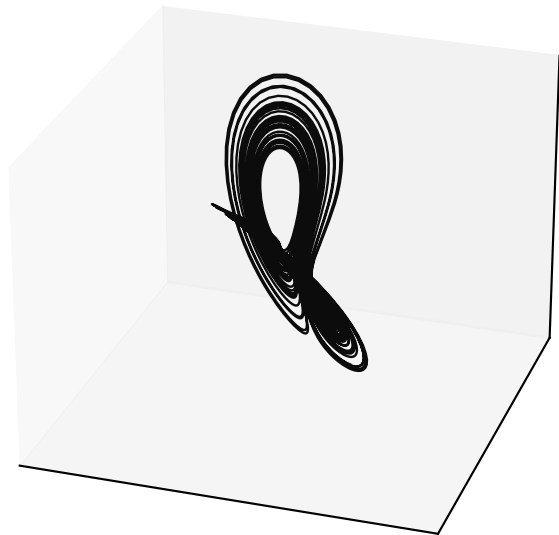


Orthogonal y-z

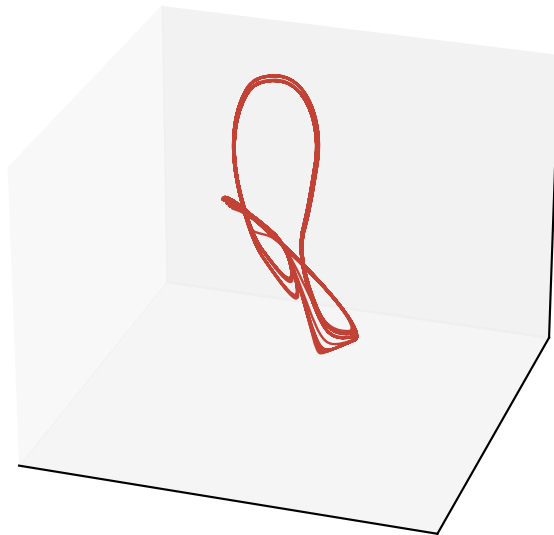


Free-run reconstruction at $P = 5$ (3-D view, 5000 steps)

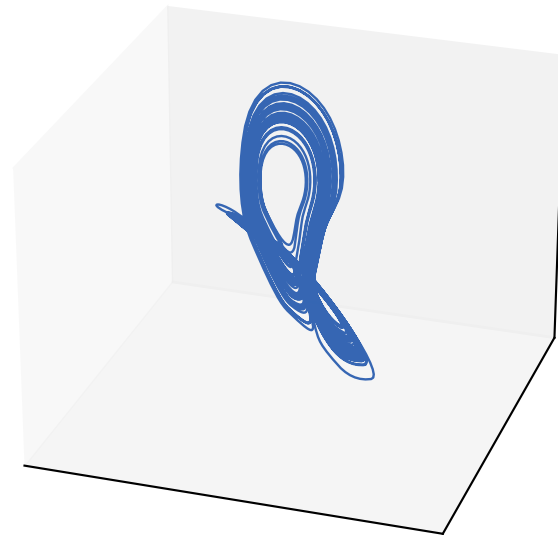
Ground truth (rotated)



Vanilla AL-RNN

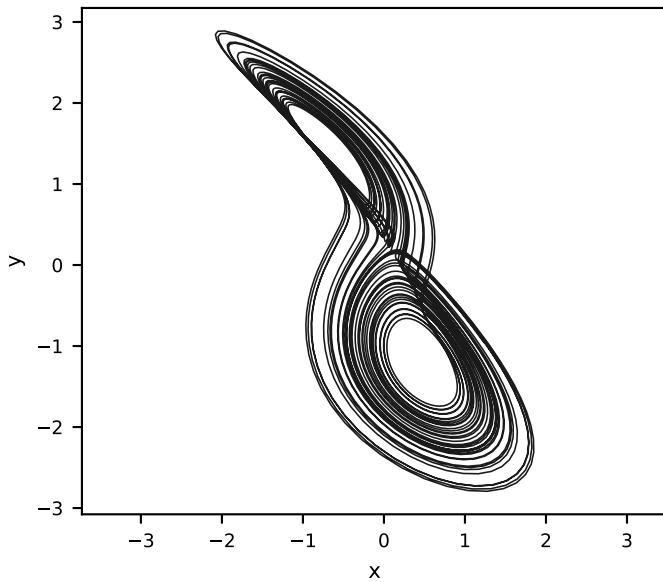


Orthogonal AL-RNN

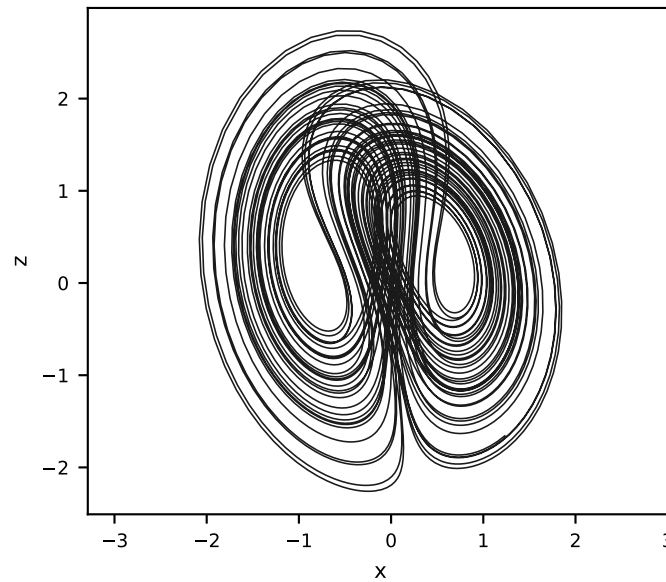


Free-run reconstruction at $P = 5$ (2-D projections, 5000 steps)

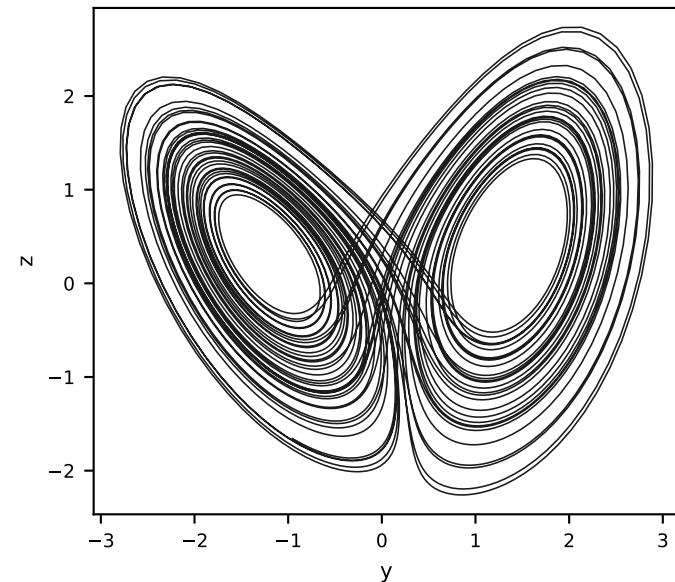
Ground truth x-y



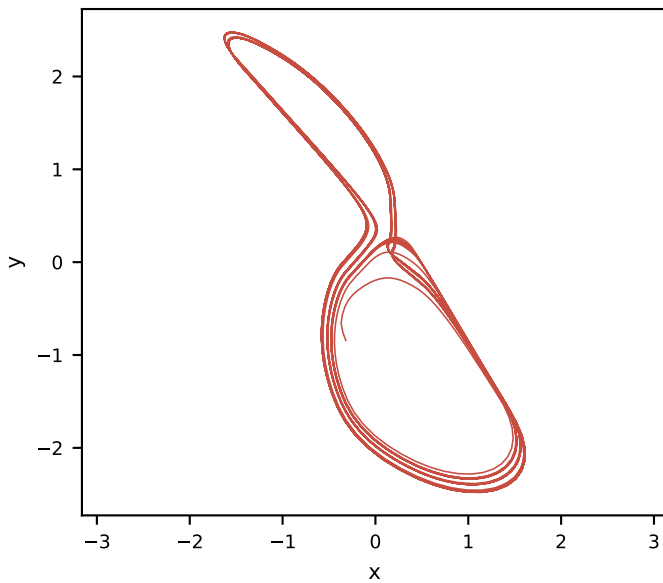
Ground truth x-z



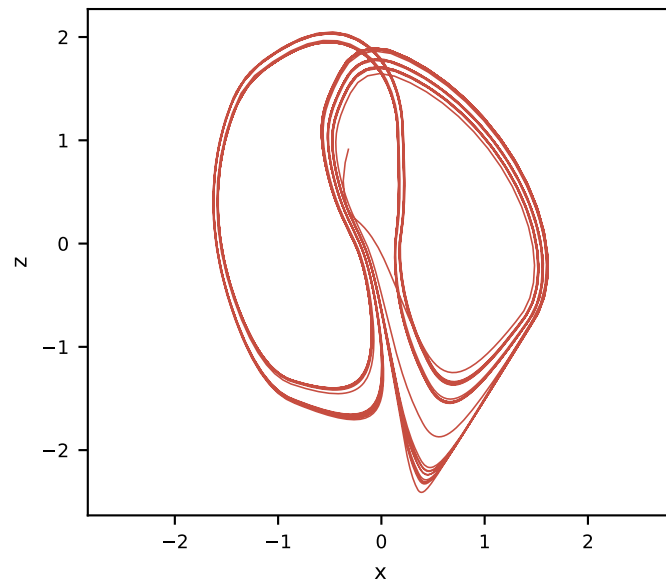
Ground truth y-z



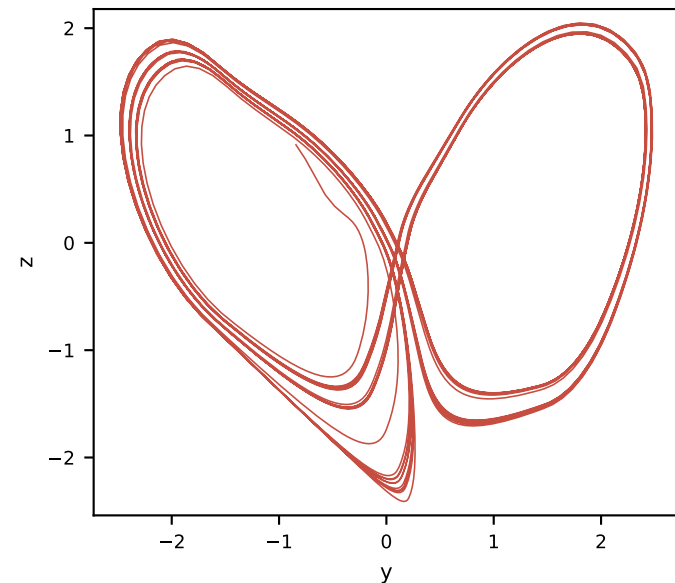
Vanilla x-y



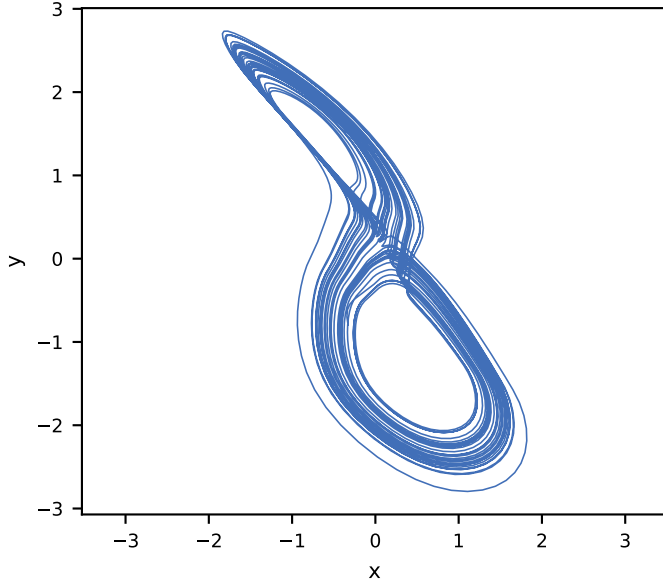
Vanilla x-z



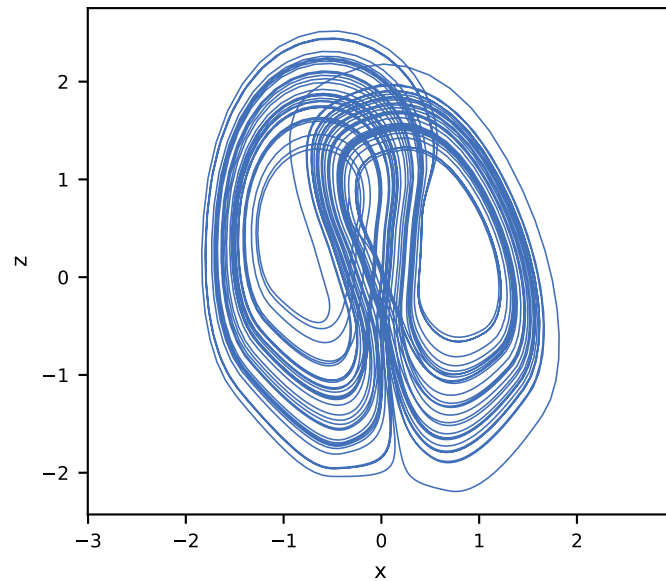
Vanilla y-z



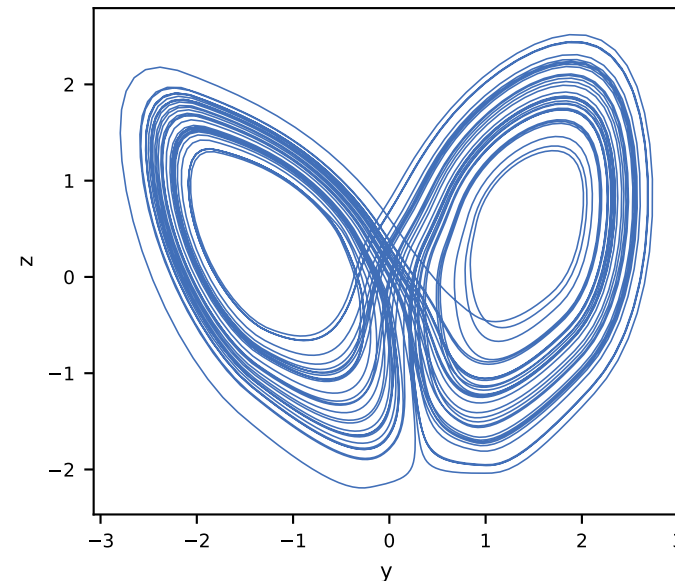
Orthogonal x-y



Orthogonal x-z

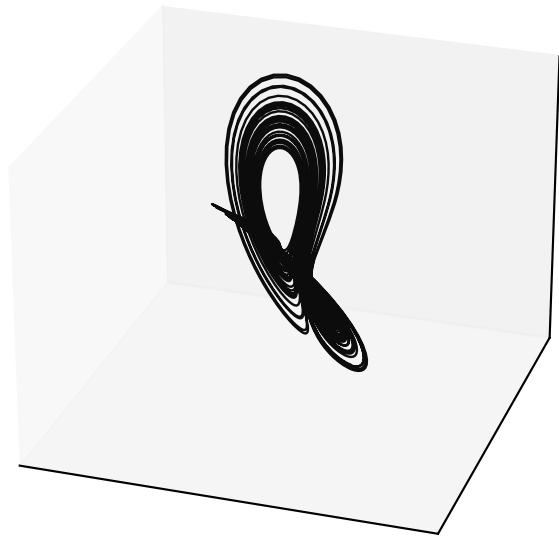


Orthogonal y-z



Free-run reconstruction at $P = 6$ (3-D view, 5000 steps)

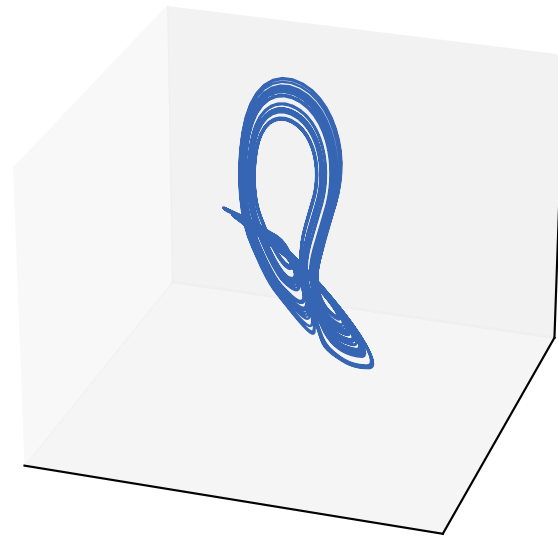
Ground truth (rotated)



Vanilla AL-RNN

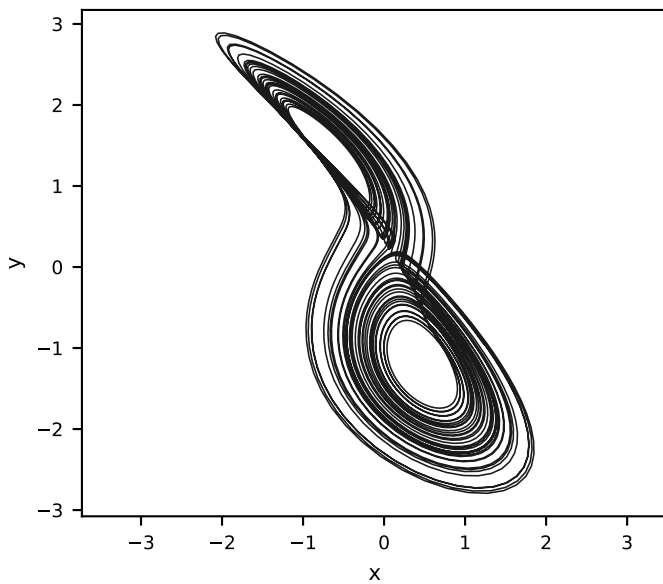


Orthogonal AL-RNN

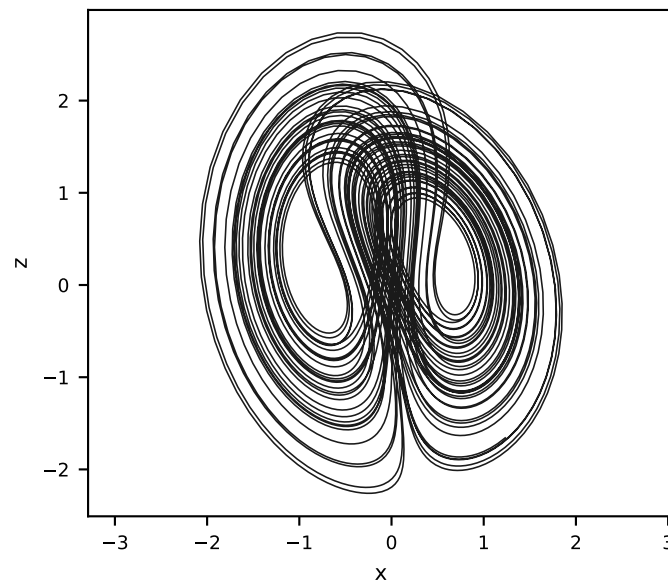


Free-run reconstruction at $P = 6$ (2-D projections, 5000 steps)

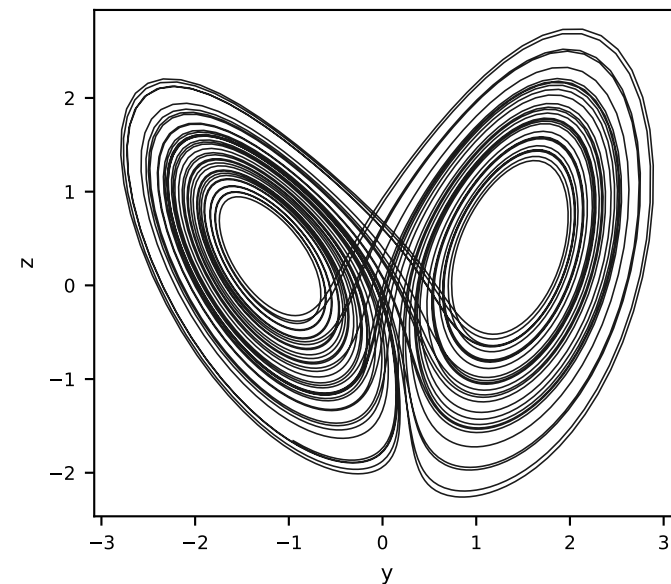
Ground truth x-y



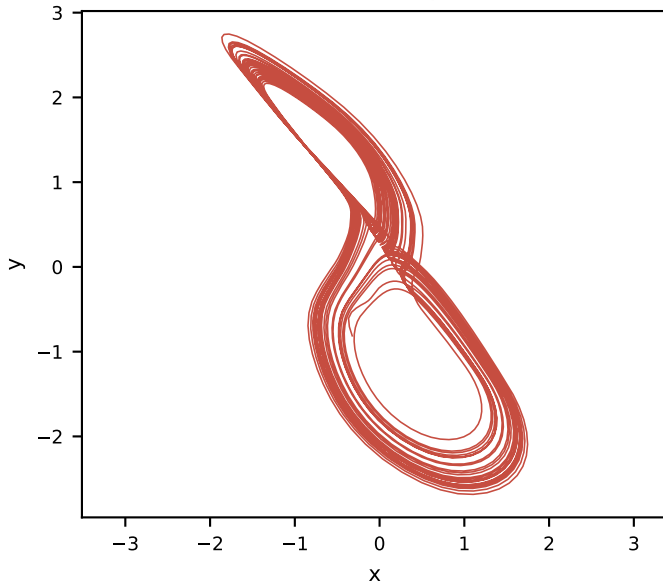
Ground truth x-z



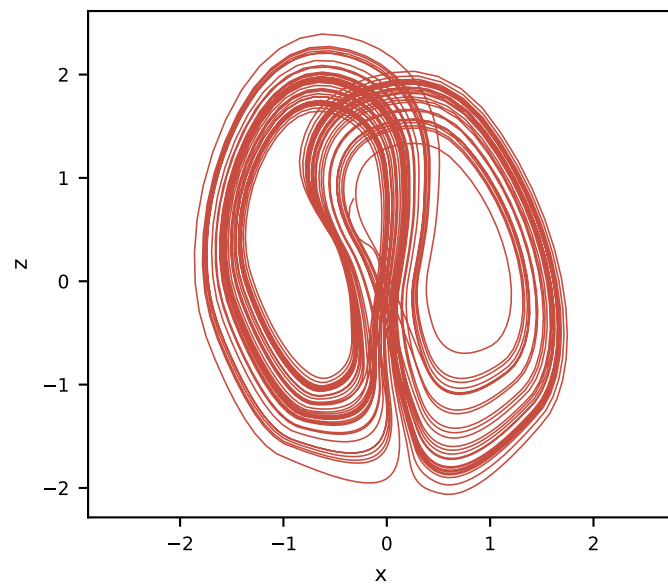
Ground truth y-z



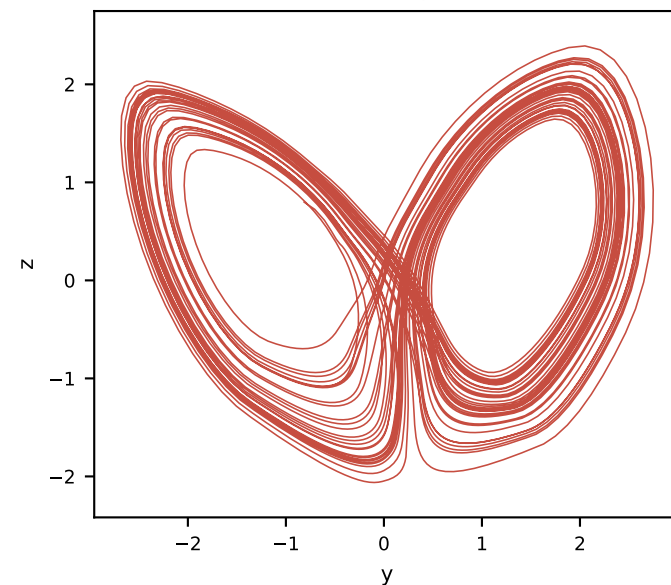
Vanilla x-y



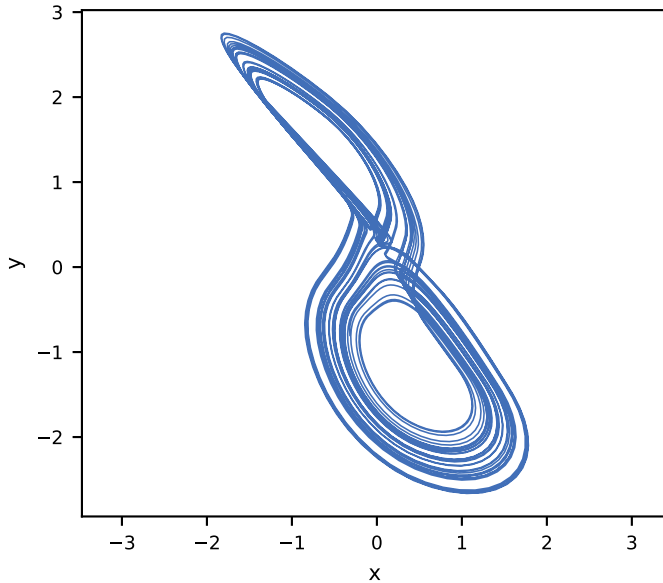
Vanilla x-z



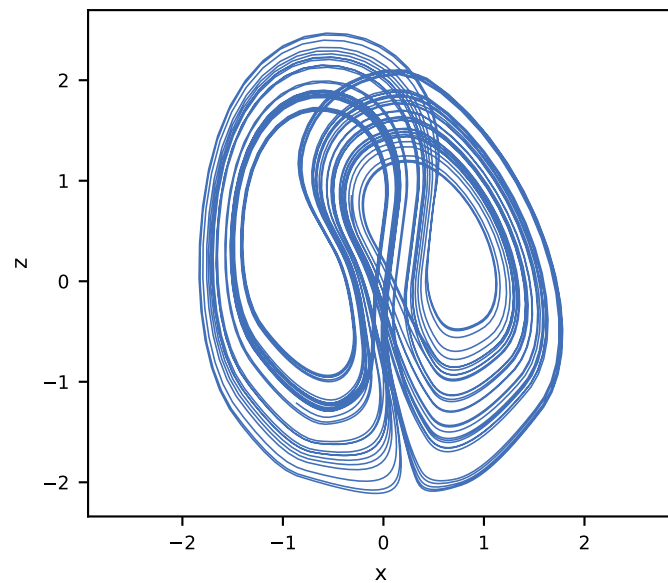
Vanilla y-z



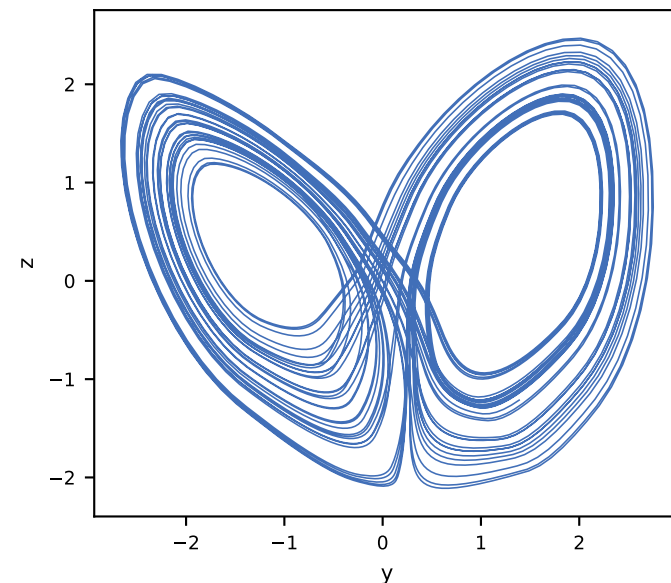
Orthogonal x-y



Orthogonal x-z



Orthogonal y-z



Learned Q (20x20) - brief diagnostics

The orthogonal constraint was enforced numerically (orthogonality error and singular-value spread are $\sim 1e-6$ across all P). For every P the learned Q is:

- * far from identity $||Q - I||_F \sim 6.1-6.5$
(random-orthogonal baseline: 6.33 ± 0.15)
- * not a permutation mean row participation $\sim 13-14$
(1 = permutation, M=20 = fully uniform mixing)
- * 9-10 complex eigenvalue pairs - i.e. ~ 9 genuine 2-plane rotations
- * $\det(Q)$ alternates ± 1 across P (rotations and rotoreflections both found; sign is init-dependent, not data-dependent)

In short, the learned Q is a dense, non-trivial full-rank rotation of the 20-d hidden state. Its role is not to 'undo the data rotation' - the 3-d observation block $Q[:3,:3]$ is far from Q_{dataset} and far from orthogonal. Q provides an extra, learned rotational degree of freedom in hidden space that improves the fit, most visibly as P grows.

Per-P numerical diagnostics (Q_analysis section of summary_main.json):

P	$ Q-I _F$	perm-res	ortho-err	mean-part	cplx-pairs	real_ ± 1	det
2	6.47	3.92	$1.6e-06$	13.83	9	2	-1.00
3	6.15	3.96	$1.8e-06$	13.80	10	0	+1.00
4	6.47	3.95	$1.6e-06$	13.50	9	2	-1.00
5	6.13	3.97	$1.5e-06$	13.80	10	0	+1.00
6	6.13	3.92	$1.6e-06$	13.20	9	2	-1.00